

清华大学本科生考试试题专页纸	
考试课程：操作系统（A）	时间：2024年04月22日上午 09:50~12:15

系别： _____	班级： _____	学号： _____	姓名： _____
--------------	--------------	--------------	--------------

答卷注意 事项：	1. 答题前，请先在试题纸和答卷本上写明A卷或B卷、系别、班级、学号和姓名。
	2. 在答卷本上答题时，要写明题号，不必抄题。
	3. 试卷有20道判断题、15道选择题、6道问答题。
	4. 请回答所有题目。本试卷共12页。最后空白页可做草稿。
	5. 考试完毕，必须将试题纸和答卷本一起交回。

一、(20分)判断题

- 1. [] 操作系统仅负责管理计算机的硬件资源。
- 2. [] 批处理系统允许多个作业同时进入内存且并行执行，作业之间不需要互相通信或协调。
- 3. [] 在拥有 U-Mode, S-Mode, M-Mode的RISC-V架构下，操作系统内核通常运行在S-mode。
- 4. [] 在有S-Mode的RISC-V中，两个进程的页表项可指向同一个物理内存页。
- 5. [] 执行系统调用可能会引起进程上下文切换。
- 6. [] OS实验中的rCore/uCore内核，每个进程的地址空间由代码段、数据段、堆段、栈段组成，这些段在物理内存中是地址连续的。
- 7. [] 在微内核操作系统中，设备驱动程序通常运行在内核空间。
- 8. [] 在非抢占调度算法中，最短作业优先(SJF)算法可以使作业的平均等待时间最小。
- 9. [] 优先级继承机制可以避免优先级反转现象。
- 10. [] 进程控制块(PCB)是操作系统管理进程的核心结构，是进程的唯一标志。
- 11. [] 通过页表机制可以实现进程地址空间之间的隔离和共享。
- 12. [] 操作系统的分页机制可有效消除内存管理中的外碎片问题。
- 13. [] 动态内存分配可提供堆与栈上的内存空间动态申请与释放服务。
- 14. [] 当持续增大时间片，时间片轮转(RR)调度算法可退化成最短剩余时间(SRT)调度算法。
- 15. [] 伙伴系统内存分配算法有助于减少内存的内碎片。
- 16. [] 最高响应比优先算法(HRRN)通过获取I/O等待时间，来动态提升等待I/O事件的GUI应用的优先级，从而降低其响应延迟时间。
- 17. [] 利用硬件中断机制，操作系统可以在用户态及时处理用户进程的异常行为。
- 18. [] 如采用覆盖(Overlay)技术，不存在调用关系的函数或模块可共

用同一块内存区域。

19. [] 一个进程在一次 exec 系统调用的前后，其PID不会发生改变。
20. [] 当操作系统检测到进程非法访问内存的异常时，一般会杀死触发异常的进程以保护系统安全。

二、(15分) 选择题

1. 操作系统内核的特征包括 ()
 - A. 并发、共享、虚拟、异步
 - B. 串行、独占、虚拟、同步
 - C. 并发、独占、物理、同步
 - D. 串行、共享、物理、异步
2. 操作系统中虚拟内存管理的主要作用是 ()
 - A. 提高CPU计算速度
 - B. 允许进程直接管理物理内存
 - C. 加速进程间的数据交换
 - D. 给进程提供独立的地址空间
3. 在uCore/rCore操作系统中，进程A切换到进程B的过程中不会涉及的操作是 ()
 - A. 处理器特权级切换
 - B. 进程之间的页表切换
 - C. 用进程B的代码段覆盖进程A的代码段
 - D. 改变进程A的运行状态
4. 在操作系统中，当一个进程发生异常并被终止时，操作系统处理该进程时不会做的步骤包括 ()
 - A. 释放分配给该进程的内存资源
 - B. 创建该进程的子进程
 - C. 回收该进程的进程控制块
 - D. 进行进程切换
5. 在某操作系统中，进程A通过系统调用向操作系统通知，自己需要等待某个时间点发生并且主动让出处理器，操作系统让进程A进入等待队列，并在就绪队列中选中进程B并调度其恢复执行。如果不考虑其他事件产生，那么在这个过程中会发生 ()
 - A. 4次特权级切换
 - B. 3次页表切换
 - C. 2次改变进程状态
 - D. 0次用户栈与内核栈切换
6. 以下关于页表机制的描述中，正确的是 ()
 - A. 虚拟页号的位数一定小于物理页号的位数
 - B. 虚拟页号的位数一定大于物理页号的位数
 - C. 虚拟页号的位数一定等于物理页号的位数
 - D. 以上选择都不对
7. 可执行程序是一种二进制文件，它包含了代码和数据，下列说法正确的是 ()
 - A. 可执行程序不需要加载到内存中就能执行

- B. 可执行程序的代码内容可被多个进程共享
 - C. 可执行程序可具有自己的页表和地址空间
 - D. 操作系统内核为每个可执行程序创建对应的进程控制块
8. 在中断处理过程中，必须保存和恢复中断上下文的原因是（ ）
- A. 防止进程抢占内核资源
 - B. 维持函数调用顺序的正确性
 - C. 提高CPU执行效率
 - D. 确保中断返回时可以恢复到正确的执行状态
9. 基于最优匹配（Best Fit）内存分配策略/算法的内存分配属于（ ）
- A. 静态连续内存分配
 - B. 静态非连续内存分配
 - C. 动态连续内存分配
 - D. 动态非连续内存分配
10. 在uCore/rCore实验中，下列哪些内容会同时位于用户地址空间和内核地址空间？（ ）
- A. 内核代码和数据段
 - B. trampoline跳板页
 - C. 固件相关的内存区域
 - D. 包含虚拟地址0的页面
11. 在面向RISC-V的操作系统设计中，如果要支持用户态程序进行系统调用，以下哪一措施是必要的？（ ）
- A. 让用户程序直接修改内核空间的地址
 - B. 让用户程序和内核共享一个页表
 - C. 通过软件中断指令（如ecall指令）从用户态切换至内核态
 - D. 使用特权模式load/store指令实现用户态到内核态的数据传递
12. 不用Python语言而用C/Rust设计实现操作系统内核的原因是（ ）
- A. Python语言是一种解释型语言，执行效率低
 - B. Python语言没有提供直接操作硬件的底层控制能力
 - C. Python语言不支持直接访问内存地址，而采用自动内存管理
 - D. 以上都是
13. 在虚拟内存管理中，将虚拟地址转换为物理地址的硬件是（ ）
- A. 算术逻辑单元（ALU）
 - B. 存储管理单元（MMU）
 - C. 页表（Page Table）
 - D. 可编程中断控制器（PIC）

14. 在虚拟内存管理中，影响缺页率的因素有（ ）
- A. 页面置换算法
 - B. 程序的执行行为
 - C. 页大小
 - D. 以上都是
15. 实时操作系统中Rate Monotonic (RM) 调度算法的基本原则是（ ）
- A. 任务优先级基于它们的执行时间
 - B. 任务优先级基于它们的等待时间
 - C. 任务优先级基于它们的执行周期
 - D. 所有任务具有相同的优先级

三、(65分)共6道问答题

1. (12分)

进程的动态执行状态（如下图所示）管理和对系统调用的支持是操作系统的重要功能。进程发出系统调用可能会影响进程的执行状态。请回答如下问题：

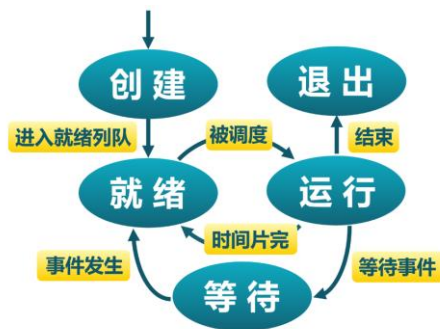


图1.1 进程的动态执行状态

1. 进程成功执行fork系统调用会导致进程的执行状态出现哪些变化。
2. 进程成功执行exec系统调用会导致进程的执行状态出现哪些变化。
3. 进程成功执行waitpid系统调用会导致进程的执行状态出现哪些变化。
4. 进程成功执行exit系统调用会导致进程的执行状态出现哪些变化。
5. 进程成功执行read系统调用会导致进程的执行状态出现哪些变化。
6. 子进程由父进程创建，但子进程的结束和父进程的运行是一个异步的过程，因此出现了“僵尸进程”和“孤儿进程”的情况。请解释“僵尸进程”和“孤儿进程”的基本概念，包括产生的原因。

注：exec系统调用是指Linux/Unix中的execve系统调用

2. (12分)

在操作系统实验环境搭建过程中，我们需要安装QEMU软件来模拟一台完整的计算机，并会执行如下的代码，来让QEMU模拟加电冷启动虚拟计算机，运行操作系统和应用：

```
qemu-system-riscv64 -machine virt -nographic -bios ../bootloader/rustsbi-qemu.bin \
    -device loader,file=target/riscv64gc-unknown-none-elf/release/os.bin,addr=0x80200000
或
qemu-system-riscv64 -nographic -machine virt -bios ../bootloader/rustsbi-qemu.bin \
    -kernel build/kernel
```

并可得到类似如下内容：

```

┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
│   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │ │   │
└───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘

[rustsbi] Implementation: RustSBI-QEMU Version 0.0.2
[rustsbi-dtb] Hart count: cluster0 with 1 cores
[rustsbi] misa: RV64ACDFIMSU
[rustsbi] mideleg: ssoft, stimer, sext (0x222)
[rustsbi] medeleg: ima, ia, bkpt, la, sa, uecall, ipage, lpage, spage (0xb1ab)
[rustsbi] pmp0: 0x100000000 ..= 0x10001fff (rwx)
[rustsbi] pmp1: 0x800000000 ..= 0x8fffffff (rwx)
[rustsbi] pmp2: 0x0 ..= 0xfffffffffffff (---)
qemu-system-riscv64: clint: invalid write: 00000004
[rustsbi] enter supervisor 0x80200000
Hello, world!
.text [0x80200000, 0x80202000)
```

请回答如下实验相关问题：

1. 上述命令中的 `../bootloader/rustsbi-qemu.bin` 和 `os.bin (kernel)`，哪个代表的是操作系统内核？哪个代表的是固件？它们分别运行在哪个特权态下？
2. 操作系统要执行的第一条指令的物理地址是？
3. 解释最后一行的“`.text [0x80200000, 0x80202000)`”的含义，并回答：操作系统建立页表时，对映射到这个范围内的页表项的R、W和X三个标识位应该如何设置？
4. QEMU模拟的是哪个指令集架构的CPU？
5. 图2.1是QEMU模拟的virt计算机物理地址空间，图2.2是对包含一小段机器码的描述。请分析reset_vec数组中的机器码，并回答：这段代码会在哪种特权模式下执行？这段代码执行完毕后，会传递什么信息给后续执行代码？后续执行代码的起始地址是？后续执行的代码是uCore/rCore操作系统内核吗？

提示：

- QEMU模拟RISC-V Virt计算机冷启动时，将跳到0x1000物理地址处执行第一条指令。
- 图2.2的第538行的代码中提到了“auipc”，这条指令的含义是，取当前指令的地址，加上左移12位的立即数，将结果保存到目标寄存器t0中。
- 图2.2的第540行的代码中提到的“mhartid”，在RISC-V架构中，mhartid是一个控制状态寄存器（CSR），存储了当前运行代码的硬件线程（hart）的ID。
- 图2.2的第548行的代码中提到的start_addr的赋值如下：
- `start_addr = virt_memmap[VIRT_DRAM].base;`

- VIRT_DRAM的信息可看图2.1中的第67行
- 图2.2的第550行的代码中提到了“dtb”是DTB (Device Tree Blob) 的内存起始地址,在QEMU模拟的RISC-V计算机中,DTB是一种用于描述硬件平台信息的数据结构,包括处理器、外设、内存等。

```

54     static const struct MemmapEntry {
55         hwaddr base;
56         hwaddr size;
57     } virt_memmap[] = {
58         [VIRT_DEBUG] = { 0x0, 0x100 },
59         [VIRT_MROM] = { 0x1000, 0x11000 },
60         [VIRT_TEST] = { 0x100000, 0x1000 },
61         [VIRT_RTC] = { 0x101000, 0x1000 },
62         [VIRT_CLINT] = { 0x2000000, 0x10000 },
63         [VIRT_PLIC] = { 0xc000000, 0x4000000 },
64         [VIRT_UART0] = { 0x10000000, 0x100 },
65         [VIRT_VIRTIO] = { 0x10001000, 0x1000 },
66         [VIRT_FLASH] = { 0x20000000, 0x4000000 },
67         [VIRT_DRAM] = { 0x80000000, 0x0 },
68         [VIRT_PCIE_MMIO] = { 0x40000000, 0x40000000 },
69         [VIRT_PCIE_PIO] = { 0x03000000, 0x00010000 },
70         [VIRT_PCIE_ECAM] = { 0x30000000, 0x10000000 },
71     };

```

图2.1 QEMU模拟的计算机内存

```

536     /* reset vector */
537     uint32_t reset_vec[8] = {
538         0x00000297, /* auipc t0, 0x0 */
539         0x02028593, /* addi a1, t0, 0x20 */
540         0xf1402573, /* csrr a0, mhartid */
541     #if defined(TARGET_RISCV32)
542         0x0182a283, /* lw t0, 24(t0) */
543     #elif defined(TARGET_RISCV64)
544         0x0182b283, /* ld t0, 24(t0) */
545     #endif
546         0x00028067, /* jr t0 */
547         0x00000000,
548         start_addr, /* start: .dword */
549         0x00000000,
550         /* dtb: DTB 放置在此处 */
551     };

```

图2.2 QEMU中的riscv_virt_board_init函数片段

3. (10分)

阅读如下Linux代码并回答问题：

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int loop = 8;
    int ready = 1;
    for(int i = 0; i < loop; i++) {
        if (ready == 1) {
            if (fork() == 0) {
                ready = 0;
            }
        } else {
            ready = 1;
        }
    }
    wait(NULL);
    printf("All done!\n");
    exit(0);
}
```

1. 上述代码涉及到哪些不同的系统调用（给出系统调用名即可）？
2. “fork” 返回值对父进程和子进程来说分别有哪些可能值，代表何种含义？
3. 假设程序执行期间 “fork” 调用的返回值总是不小于0，程序运行完成后总计会输出多少行 “All done!” ？
4. 请分析上述程序运行时是否可能出现父进程比子进程先退出的情况？如果有可能，请说明为确保父进程比子进程后退出该如何修改代码（说明思路即可）。
5. OS实验的uCore/rCore实验框架是单核处理器上运行的，请阐述在uCore或rCore中是如何处理并设置 “fork” 后两个进程的运行状态的？

提示：

printf信息：int printf(const char *format, ...)

wait信息：pid_t wait(int *wstatus)

exit信息：void exit(int status)

fork信息：pid_t fork(void)

4. (14分)

RISC-V使用 Sv39 页表系统，将39位的虚拟地址转换为56位的物理地址。已知 Sv39 使用3级页表，页大小 4KB。

1. 已知 Sv39 的每一级页表的页表项数量都相同，请计算（给出计算思路）：
 - a. Sv39 的每一级页表的页表项数量是多少？
 - b. 已知 Sv39 中页表大小为一个页，则每一个页表项的大小是多少？
 - c. Sv39的每一个页表项中，至少需要有多少位表示物理页号？还剩多少位可以用来存储各类标识位？
2. Sv39 的页表项中有两个标识位：访问位（A）和修改位（D）。简要介绍这两个位在页面置换过程中能够起到的作用。
3. 如果忘记清空 TLB，是否有可能出现以下错误？说明原因：
 - a. 在页表中有对应项的时候，错误地报告缺页；
 - b. 在页表中没有对应项的时候，错误地不报告缺页；
 - c. 访存指令读取到错误的数据。
4. 更改页表和修改页表之后要及时刷新 TLB。故某同学在完成lab2（第四章）实验时，在 `mmap` 和 `munmap` 函数中加入了大量的刷新 TLB 的语句。请回答如下实验相关问题：
 - a. Lab2（实验指导书第四章）实验中何时更换页表？如何更换页表？
 - b. 该同学插入的语句有必要吗？为什么？
 - c. 更换页表之后需要刷新 cache 吗？无需解释原因。
 - d. 请解释trampoline的作用，以及其物理地址与虚拟地址映射的特点。
5. 我们课程实验采用了所谓的“双页表”方案，即用户程序和内核基于不同的页表，在不同的地址空间。假设我们要实现“单页表”方案，即进程和内核共用一个页表，让内核占据地址空间的高半部分。请问：
 - a. rCore中采用MapType的Identical类型来建立地址空间，或者uCore中采用“物理地址==虚拟地址”的方式对于内核还适用吗？
 - b. 如果适用，请解释理由。如果不适用，请设计一种适用“单页表”的地址转换方案，使得在内核中使用的虚拟地址和物理地址可以相互转化。你的设计只需对内核适用，补全下面的函数即可。（注：你在这两个函数中无需也不应该查页表）

// 可在函数中添加内容

Rust:

```
fn virt_addr_to_phys_addr(va: usize) -> usize { ... }
```

```
fn phys_addr_to_virt_addr(pa: usize) -> usize { ... }
```

或

C:

```
uint64 virt_addr_to_phys_addr(uint64 va) {...}
```

```
uint64 phys_addr_to_virt_addr(uint64 pa) {...}
```

5. (8分)

- 对于通用操作系统而言，当操作系统的异常处理例程执行完毕之后，以下各种情况是否可能发生？如果可能，举一个例子，如果不可能，说明原因：
 - 回到产生异常的进程，执行产生异常的指令的下一条指令；
 - 回到产生异常的进程，重新执行产生异常的指令；
 - 直接结束产生异常的进程。
- 以下是 uCore/rCore 内核的Trap处理函数的一部分（有改动），请根据代码内容回答如下问题。

```
/// trap handler
#[no_mangle]
pub fn trap_handler(cx: &mut TrapContext) -> &mut TrapContext {
    let scause = scause::read(); // A
    let stval = stval::read();
    match scause.cause() {
        Trap::Exception(Exception::UserEnvCall) => {
            cx.sepc += 4; // B
            cx.x[10] = syscall(cx.x[17], [cx.x[10], cx.x[11], cx.x[12]]) as usize;
        }
        Trap::Exception(Exception::StoreFault) |
        Trap::Exception(Exception::StorePageFault) => {
            println!("??????, ?? = {:#x}, pc = {:#x}.", stval, cx.sepc); // C
            exit_current_and_run_next();
        }
        _ => {
            panic!("Unsupported trap {:?}, stval = {:#x}!", scause.cause(), stval);
        }
    }
    cx
}
```

```

/// trap handler
void usertrap() {
    struct trapframe *trapframe = curr_proc()->trapframe;

    uint64 cause = r_scause(); // A
    switch (cause) {
    case UserEnvCall:
        trapframe->epc += 4; // B
        syscall();
        break;
    case StoreMisaligned:
    case StorePageFault:
        printf("?????, ?? = %p, pc = %p.\n", cause, r_stval(), trapframe->epc); // C
        exit(-2);
        break;
    default:
        unknown_trap();
        break;
    }
    usertrapret();
}

```

- 代码中A处读取到的是什么信息？简述其内容；
- 代码中B处为什么要将这个字段的值加上4？说明理由；
- 代码中C处的日志信息应该报告一个什么错误？stval保存的是什么信息？目前出现此异常时，操作系统会直接结束当前进程，未来利用此异常可以实现什么功能？

6. (9分)

在13个时间单位的时间内，一个程序的虚拟页面访问顺序如下：

1	2	3	4	5	6	7	8	9	10	11	12	13
a	d	e	c	c	d	c	b	e	c	e	a	e

初始物理页帧为空。请回答如下问题：

1. 请描述 LRU、Clock、工作集、缺页率页面置换算法的基本思路和特征。
2. 为何 LRU 算法难以在实际操作系统中实现？
3. 为何 LRU 算法不会出现 Belady 现象？
4. 如采用工作集页面置换算法，工作集窗口 Δ 的大小 $\tau = 4$ ，请问在哪些时刻会产生缺页异常，缺页异常次数总共是多少？
5. 如采用缺页率置换算法，容忍的缺页窗口 $T=3$ ，请问在哪些时刻会产生缺页异常，缺页异常次数总共是多少？

